

面向对象程序设计

主要内容

- 类和对象
- 封装
- 继承
- 魔术方法

对象和类(Object and Class)

- 对象：客观世界中存在的事物的抽象
 - 包含状态（数据字段或**属性**）和行为（动作或**方法**）的编程实体
 - 状态：存储在对象中的一组值（内部数据）
 - 行为：对象可以执行的操作集合，比如显示或者修改其内部的状态
 - python语言中所有的数据（包括函数等）都是对象
- 类：描述了同一类别的对象的共同特征
 - 类给出了描述某种类型的对象的状态和行为的**模板**(template或blueprint)
 - 类给出了如何创建该类的对象、该类的对象所拥有的状态以及可以执行的行为
 - 类的实例对象(instance object)：使用类创建的该类型的对象
 - 通过class语句定义类
 - 类本身也是一个对象，称为**类对象**
 - 之前介绍的int/float/str/list/tuple/dict/set都是类对象
- 对象的客户端：创建和使用对象的代码

对象和类(Object and Class)

- 类名建议采用每个单词首字母大写的CamelCase命名风格（许多内置的类型如str采用小写单词风格）
- 在通过class语句定义类型时：
 - **创建一个类对象**，执行class语句中class body的代码。关于名字空间的概念将在后面的示例中说明
 - 如果第一个语句为字符串字面量定义，则保存在该类对象的`__doc__`属性中
 - `class body`不是一个函数名字空间，而是一个特殊的本地名字空间
 - 在执行过程中引入的没有限定的名字属于**该类对象中的属性和方法**
 - 该非限定名的作用域范围**不会扩展到方法中**
 - 方法中的名字遵循LEGB规则解析时，不会考虑class body的赋值类语句引入的未限定名
 - 解析未限定名，遵循LEGB规则：**本地名字空间(类对象的属性和方法)** --> 外层名字空间
 - 在**当前名字空间引入一个名字ClassName**，该名字指向新创建的类对象
- 类实例化：将类对象看成函数的形式，通过调用创建一个新的实例对象

```
class ClassName:  
    class body  
  
instance = ClassName(arguments)
```

```
class Date:  
    """ A Date object represents a month and day such as 12/25.  
        First version: a completely empty class.  
    """  
    pass  
  
d1 = Date()  
print(Date.__doc__)
```

date1.py

A Date object represents a month and day such as 12/25.
First version: a completely empty class.

class语句：__init__和__str__方法

- 客户端可以访问和修改（新增）实例对象的属性
- class语句体定义了为该类的实例对象所提供的行为
 - 缺省继承了**object类对象**，其中给出了__init__，__str__等方法的缺省实现
 - 构造函数__init__：在实例化对象时调用，以对该对象进行初始化
 - __str__方法：在调用内置函数str(obj)时调用，返回一个用户友好的字符串

```
class Date:
    """ A Date object represents a month and day such as 12/25.
        First version: a completely empty class.
    """
    pass
```

```
d1 = Date(12, 25)
d2 = Date(2, 14)
print(d1)
print(d2)
```

对象：

- 状态（数据字段或**属性**）
和行为（动作或**方法**）
- 对象使用者不用关心内部的细节

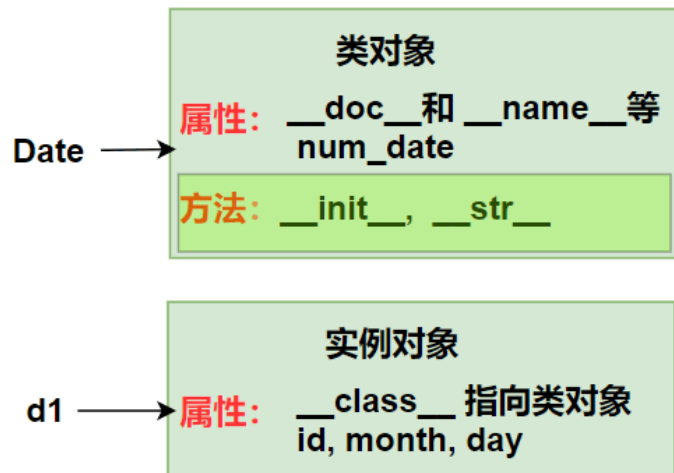
```
d1 = Date()
d1.month = 12
d1.day = 25
d2 = Date()
d2.month = 2
d2.day = 14
print('{:02}-{:02}'.format(d1.month, d1.day))
print('{:02}-{:02}'.format(d2.month, d2.day))
```

class语句: `__init__`和`__str__`方法

- 构造函数: 在创建类型的实例对象时, 对该实例对象进行初始化

- 在调用构造函数之前, 已经创建了一个实例对象
- `__init__`函数的第1个参数就是该实例对象, 后面可有0到多个参数
- 构造函数的返回值必须是None

- 调用`str(obj)`会调用`type(obj)`的`__str__`方法, 第1个参数是实例对象obj



date2.py

```
class Date:
    """ A Date object represents a month and day.
    """
    num_date = 1 # 类属性: 记录了下一个Date对象的ID, 从1开始

    def __init__(self, month=1, day=1):
        self.month = month
        self.day = day
        self.id = Date.num_date # 使用num_date会出现NameError异常
        Date.num_date += 1

    def __str__(self):
        return '[{}] {:02}-{:02}'.format(self.id, self.month, self.day)

d1 = Date(12, 25)
d2 = Date()
print(d1)
print(d2)
```

```
[1] 12-25
[2] 01-01
```

类对象的属性和方法

通过class语句创建一个类对象，执行class语句中class body的代码

- class body执行创建一个特殊的本地名字空间，不是函数名字空间
- 执行过程中赋值类语句**引入的没有限定的名字属于该类对象中的属性和方法**
- 引入的未限定名的作用域仅在class body，不会扩展到class body定义的方法中
- 表达式出现的未限定名遵循LEGB顺序查找：**本地名字空间(类对象的属性和方法)** --> 外层名字空间
- class body通过def语句定义的方法：
 - 方法 = 特殊的函数
 - 遵循函数名字空间的相关规则
 - class body引入的名字空间不是函数空间，无法在方法内直接访问
 - **方法的外层名字空间为class语句所在的名字空间**
 - 解析未限定名时不会考虑class body的赋值类语句引入的未限定名（即类对象的属性和方法）

```
>>> Date.__str__(d1)
'[1] 12-35'
```



date2.py

```
class Date:
    """ A Date object represents a month and day.
    """
    num_date = 1 # 类属性: 记录了下一个Date对象的ID, 从1开始

    def __init__(self, month=1, day=1):
        self.month = month
        self.day = day
        self.id = Date.num_date # 使用num_date会出现NameError异常
        Date.num_date += 1

    def __str__(self):
        return '[{}] {:02}-{:02}'.format(self.id, self.month, self.day)

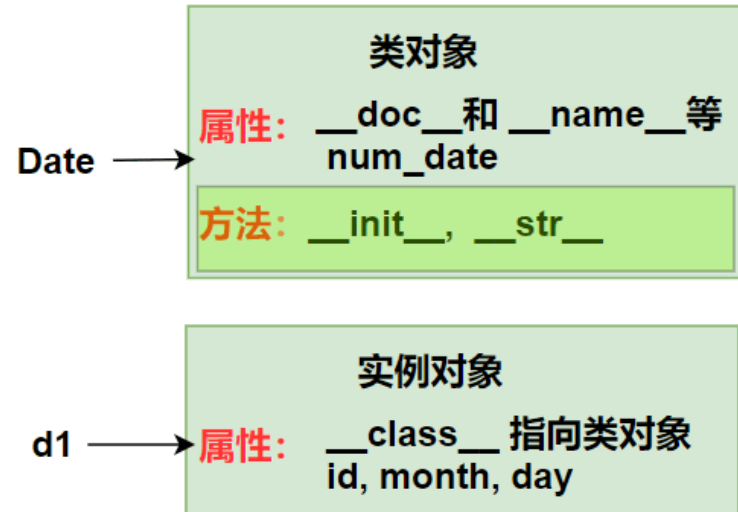
print(Date.num_date)
print(Date.__str__)
```

1

<function Date.__str__ at 0x00000206823804C0>

实例对象的属性和方法

- 每个实例对象都独立存储自己的属性数据，缺省保存在__dict__中
- instance.attr: 访问实例对象instance中名字为attr的属性
- instance.attr = value: 添加新的属性或者更改已有的属性
- **拓展: class body中 __slots__ = 'x', 'y'**
 - 限定实例对象只有x和y两个属性，更少的空间和更快的属性访问
- 实例对象的方法是bound method
 - 要执行的代码来自于类对象
 - 在class语句中通过def语句定义实例方法，该方法的第一个参数建议取名为self，表示实例对象
 - 在通过instance.method()调用时，不需要传递实例对象，解释器会将当前实例对象作为第一个参数
 - 通过Class.method()调用时，必须传递实例对象



date2.py

```
class Date:
    num_date = 1 # Date对象的个数
    def __init__(self, month=1, day=1):
        self.month = month
        self.day = day
        self.id = Date.num_date
        Date.num_date += 1

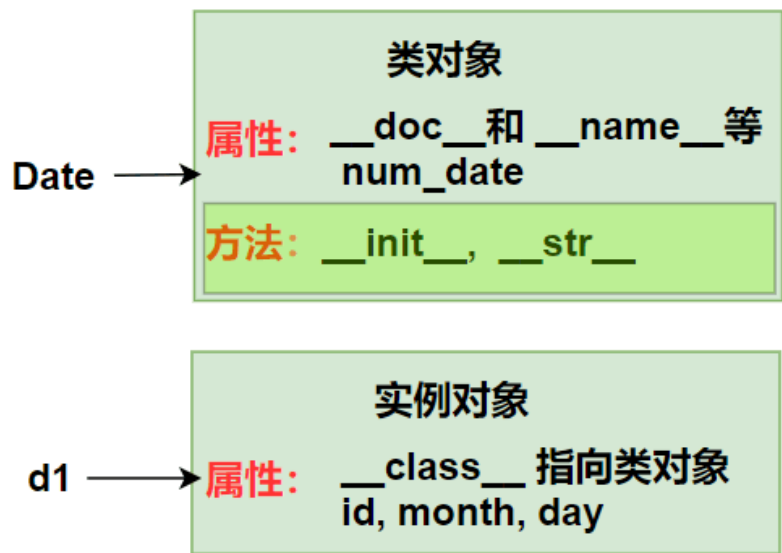
    def __str__(self):
        return '[{}] {:02}-{:02}'.format(self.id, self.month, self.day)

d1 = Date(12, 25)
```

```
>>> d1.__str__()
'[1] 12-25'
>>> Date.__str__(d1)
'[1] 12-25'
```

访问哪个对象的属性或方法？ IC (Instance, Class)

- instance.attr:
 - 访问实例对象instance中名字为attr的属性
 - 如果实例对象没有名为attr的属性时，会在其对应的类对象中查找是否有名为attr的类属性。如果不存在时，在PO (Parent, Object)中查找，称为**ICPO**，在介绍继承时展开
- instance.attr = value: 改变的是实例对象instance的名为attr的属性
- Class.attr = value: 改变的是类对象Class的名为attr的属性
- Class.attr: 返回类对象中名字为attr的属性。如果不存在时，在PO中查找，称为**CPO**



```
>>> d1 = Date()
>>> d1.num_date
3
>>> Date.num_date
3
>>> d1.__str__ is Date.__str__ # why?
False
```

实例方法、类方法和静态方法

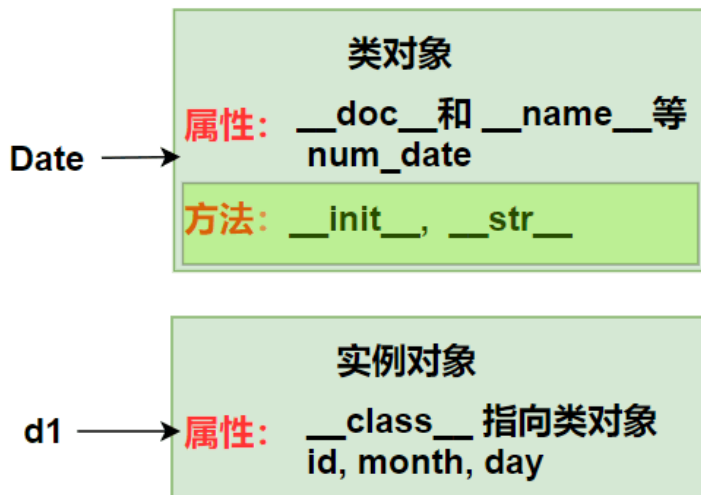
- class语句中定义的方法缺省为实例方法：
 - 通过instance.attr(...)调用实例方法时，**第一个参数为已经绑定的实例对象instance**
 - 通过ClassName.attr(...)调用实例方法时，需要自己传递某个实例对象

- 可通过装饰器classmethod声明某个方法为类方法
 - 在调用该方法时，隐藏了**第1个参数为类对象**，方便访问类对象的属性

- 可通过装饰器staticmethod声明某个方法为静态方法

- 在调用该方法时，没有隐藏任何参数
- 如同普通函数一样调用

```
>>> c1 = Circle()
>>> c1.get_perimeter
<bound method Circle.get_perimeter of
<__main__.Circle object at 0x05B33F50>>
>>> c1.get_cid
<bound method Circle.get_cid of <class '__main__.Circle'>>
>>> c1.helper
<function Circle.helper at 0x0000028716434A60>
```



Circle.py

```
class Circle:
    cid = 0
    def __init__(self, radius=1):
        self.radius = radius
        Circle.cid += 1
        self.cid = Circle.cid

    def get_perimeter(self):
        return Circle.helper(self.radius)

    @classmethod
    def get_cid(cls):
        return cls.cid

    @staticmethod
    def helper(radius):
        return 2 * math.pi * radius
```

主要内容

- 类和对象
- **封装**
- 继承
- 魔术方法

访问器和赋值器 (Accessor and Mutator)

- 访问器:
 - 提供有关对象状态的信息, 但不进行状态修改的方法
 - 其他程序语言建议通过`get_xxx(self)`获得属性xxx的值
- 赋值器:
 - 修改对象内部状态的方法
 - 其他程序语言建议通过`set_xxx(self)`修改属性xxx的值

date3.py

```
def days_in_month(self):  
    if self.month in {4, 6, 9, 11}:  
        return 30  
    elif self.month == 2:  
        return 28  
    else:  
        return 31
```

```
def get_day(self):  
    return self.day  
  
def set_day(self, day):  
    self.day = day
```

```
d1 = Date(12, 25)  
print(d1)  
d1.advance()  
print(d1)
```

```
[1] 12-25  
[1] 12-26
```

```
def advance(self):  
    """ Advances the Date's state to the next day  
    """  
  
    self.day += 1  
    if self.day >= self.days_in_month():  
        # wrap to next month  
        self.month += 1  
        self.day = 1  
    if self.month > 12:  
        # wrap to next year  
        self.month = 1
```

封装(Encapsulation)

- 封装：对对象外部（客户端）**隐藏对象内部的实现细节**
 - 把客观事物抽象化成对象，专注于基本属性而非内在的细节
 - 客户端不能直接访问或修改其内部细节
- 有些程序语言支持私有属性，只允许类中的方法访问这些属性，客户端无法直接访问
 - 可根据需要提供相应的访问器和赋值器
- python语言中，**对象的属性都是公有的**
 - 客户端都可以直接访问和修改这些属性
 - "准"私有属性：约定（但仅仅是约定）特定的属性名称为私有属性
 - 属性名以下划线开始，不建议客户端直接访问
 - 属性名(__Attr)以两个下划线开始，不以两个下划线结束，名字改为：**ClassName__Attr**
 - 对象内部对于__Attr的访问都被更改为对于新的名字的属性的访问
 - 客户端无法通过属性__Attr访问

```
def get_day(self):  
    return self.__day  
  
def set_day(self, day):  
    self.__day = day
```

member.py

```
class C:  
    def __init__(self, value=0):  
        self._value = value  
        self.__value1 = value  
        self.__value2_ = value  
        self.__value3__ = value
```

```
obj = C()  
print(dir(obj))
```

```
['_C__value1', '_C__value2_', ..., '__value3__', '_value']
```

封装：属性方法(property)

- 对象的方法中应该考虑传递的参数可能不合法的情形
- python语言中，对象的属性都是公有的，客户端可以直接访问和修改这些属性
- 要考虑客户端修改属性时设置为不合法的值，引入属性方法property

date4.py

```
class Date:
    num_date = 1 # Date对象的个数

    def __init__(self, month=1, day=1):
        self._month = month
        self._day = day
        self._id = Date.num_date
        Date.num_date += 1

d1 = Date(12, 32) # 应该检查错误
d1._day = 32     # 如何避免设置为不合适的值?
```

```
class Date:
    num_date = 1 # Date对象的个数

    def __init__(self, month=1, day=1):
        if month >= 1 and month <= 12:
            self._month = month
        else:
            raise ValueError("Invalid month: " + str(month))
        if day >= 1 and day <= self.days_in_month():
            self._day = day
        else:
            raise ValueError("Invalid day: " + str(day))
        self._id = Date.num_date
        Date.num_date += 1
```

封装：属性方法(property)

python引入了称为属性(property)的**特殊方法**

- 用于访问对象的数据属性(attribute)
- 调用它的语法与直接访问数据属性的语法完全相同
- 通过装饰器property实现，仅通过装饰器property引入的**property只读**

```
class ClassName:
    @property
    def property_name(self):
        # code to return the property's value
        return 0
obj = ClassName()
print(obj.property_name)
```

```
class Date:
    def __init__(self, month=1, day=1):
        self.set_date(month, day)

    @property
    def month(self):
        return self._month

    def set_month(self, month):
        self.set_date(month, self._day)

    @property
    def day(self):
        return self._day
```

date5.py

```
def set_day(self, day):
    self.set_date(self._month, day)

def set_date(self, month, day):
    if month >= 1 and month <= 12:
        self._month = month
    else:
        raise ValueError()
    if day >= 1 and day <= self.days_in_month():
        self._day = day
    else:
        raise ValueError()
```

```
d1 = Date(12, 27)
print(d1.month, d1.day)
d1.day = 28 # AttributeError: can't set attribute
```

封装：属性方法(property)

python支持属性赋值器，允许如同普通的数据属性一样，修改property的值

- 装饰器property将某个方法(name)改造为只读property
- 装饰器 name.setter将某个方法改造为读写的property

```
class ClassName:
    @property
    def property_name(self):
        # code to return the property's value
        return 0

    @property_name.setter
    def property_name(self, arguments):
        # code to set the property's value
        print(arguments)
```

```
obj = ClassName()
print(obj.property_name)
obj.property_name = 'set'
```

0
set

```
d1 = Date(12, 25)
print(d1)
print(d1.day)
d1.day = 28
print(d1)
```

date6.py

```
class Date:
    def __init__(self, month=1, day=1):
        self.set_date(month, day)

    def __str__(self):
        return '{:02}-{:02}'.format(self._month, self._day)

    @property
    def month(self):
        return self._month

    @month.setter
    def month(self, value):
        self.set_date(value, self._day)

    @property
    def day(self):
        return self._day

    @day.setter
    def day(self, value):
        self.set_date(self._month, value)
```

封装：属性方法(property)

- 装饰器实现property
- property(fget=None, fset=None, fdel=None, doc=None) 构造一个property, 在获得、设置或者删除该property会调用相应的方法fget/fset/fdel, doc为该property的文档字符串

```
class Date:
    def __init__(self, month, day):
        self._month = month
        self._day = day

    def get_month(self):
        return self._month

    def set_month(self, value):
        # check if value is valid
        self._month = value

    month = property(get_month, set_month, None, "Month Property")

    def get_day(self):
        return self._day

    def set_day(self, value):
        # check if value is valid
        self._day = value

    day = property(get_day, set_day, None, "Day Property")
```

```
d1 = Date(12, 27)
print(d1.month, d1.day)
d1.day = 28
```

```
>>> help(Date)
...
| Data descriptors defined here:
| day
|   day property
|
| month
|   month property
```

date7.py

主要内容

- 类和对象
- 封装
- **继承**
- 魔术方法

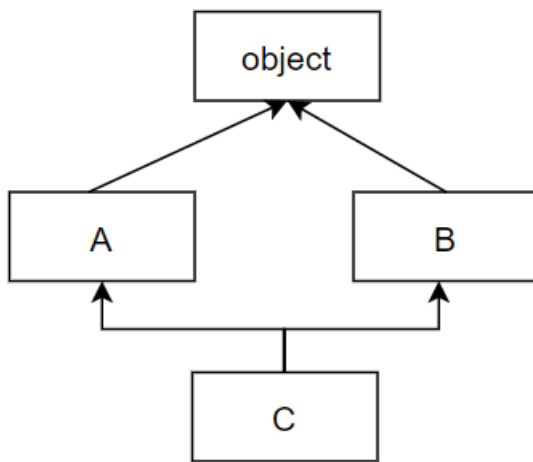
继承 (inheritance)

- 继承允许使用某个类的功能，并在无需重新改写原来的类的情况下，对这些功能进行扩展
- 被继承的类称为基类(base class)/超类(super class)或父类(parent class)，通过继承创建的新类称为子类(subclass)或派生类(derived class)
- python提供了object类，实现了一些最基本的方法（比如__str__、__repr__、__format__等方法），我们之前介绍的class语句中没有指定父类，隐含着其父类为object
- 继承实现了is-a关系：car是一种vehicle，flower是一种plant
- 一个子类一般继承一个父类，但也允许继承多个父类，class C(A, B): 表示继承A和B

```
class A(object):  
    pass
```

```
class B(object):  
    pass
```

```
class C(A, B):  
    pass
```



```
class ClassName:  
    class body
```

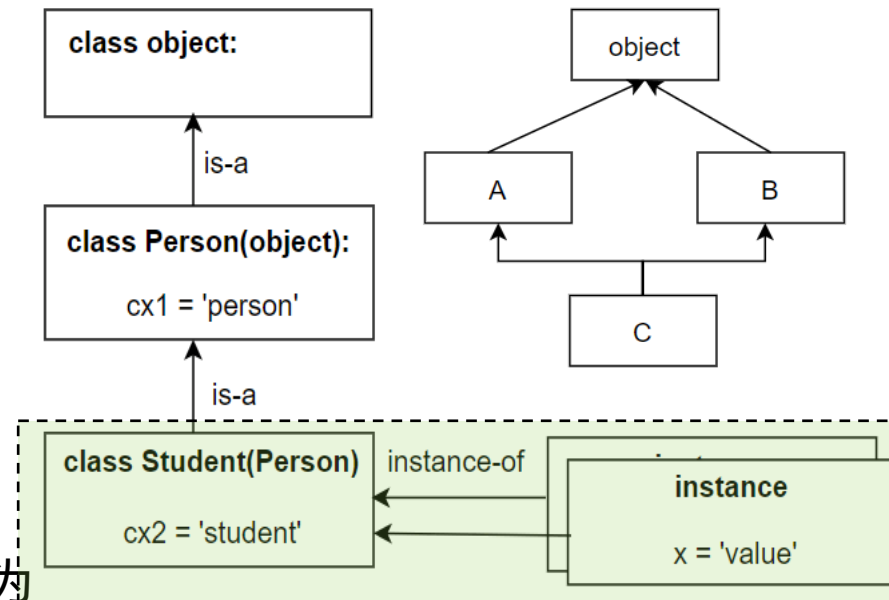
```
class ClassName(argument_list):  
    class body
```

第1种格式相当于如下第2种格式

```
class ClassName(object):  
    classbody
```

继承：属性访问顺序（ICPO, Instance/Class/Parent/Object）

- 属性出现在赋值语句的左边：
 - `instance.attr = value`, 修改实例对象instance的名字为attr的属性
 - `class1.attr = value`, 修改类对象class1的名字为attr的属性
- 属性出现在表达式中, 比如`instance.attr`:
 - 首先在实例对象instance中查找, 如果没有, 则
 - 继续在实例对象的类对象中查找, 如果没有, 则
 - 继续在该类对象的祖先类中查找
 - 如果单继承, 则父类、父类的父类等等, 直到找到相应的属性为止
 - 如果多继承, 祖先类的先后顺序由C3线性化算法决定
 - 该算法确定的先后顺序可以通过**类对象的mro** (method resolution order)方法得到
 - 子类总是在父类之前; 如果有多个父类, 这些父类的先后顺序与定义时给出的顺序一致
 - 如果没有找到, 则在object中查找, 如果没有, 抛出异常AttributeError
- 如果通过class对象访问属性, 比如`cls.attr`, 则遵循CPO的搜索顺序



```
class A(object):  
    pass  
  
class B(object):  
    pass  
  
class C(A, B):  
    pass  
print(C.mro())
```

mro.py

继承：属性访问顺序(ICPO)

Animal Type: Dog
Dog is a warm-blooded animal.
Dog has four legs.

- 子类实现的某个方法一般是在其Parents类的基础上进一步实现的
 - 一种实现：首先调用其Parents的方法，然后针对子类的具体情况增加实现
 - 如果是单继承，那么调用往父类方向的某个类的相应方法
 - 如果是多继承，那么调用哪个父类的相应方法呢？
 - **super(): 调用继承顺序中的某个父辈类的相应方法**
 - 不需要显式地指出父类的名字
 - 按照MRO的顺序，从当前类往后寻找，直到发现有一个类有一个相同名字的属性为止

Animal Type: Dog
Dog is a warm-blooded animal.
Dog has four legs.

super1.py

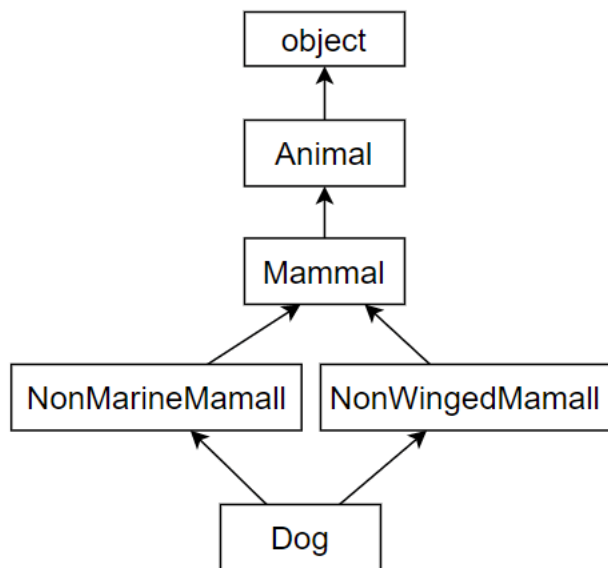
```
class Animal(object):
    def __init__(self, animal_type):
        print('Animal Type:', animal_type)

class Mammal(Animal):
    def __init__(self, mammalName):
        super().__init__(mammalName)
        print(mammalName, 'is a warm-blooded animal.')

class Dog(Mammal):
    def __init__(self):
        super().__init__('Dog')
        print('Dog has four legs.')

if __name__ == '__main__':
    dog = Dog()
```

继承：属性访问顺序(ICPO)



```
[<class '__main__.Dog'>, <class  
'__main__.NonMarineMammal'>, <class  
'__main__.NonWingedMammal'>, <class  
'__main__.Mammal'>, <class  
'__main__.Animal'>, <class 'object'>]
```

```
Dog is an animal.  
Dog is a warm-blooded animal.  
Dog can't fly.  
Dog can't swim.  
Dog has 4 legs.
```

```
class Animal:
    def __init__(self, Animal):
        print(Animal, 'is an animal.');
```

super2.py

```
class Mammal(Animal):
    def __init__(self, mammalName):
        super().__init__(mammalName)
        print(mammalName, 'is a warm-blooded animal.')
```

```
class NonWingedMammal(Mammal):
    def __init__(self, NonWingedMammal):
        super().__init__(NonWingedMammal)
        print(NonWingedMammal, "can't fly.")
```

```
class NonMarineMammal(Mammal):
    def __init__(self, NonMarineMammal):
        super().__init__(NonMarineMammal)
        print(NonMarineMammal, "can't swim.")
```

```
class Dog(NonMarineMammal, NonWingedMammal):
    def __init__(self):
        super().__init__('Dog')
        print('Dog has 4 legs.');
```

```
d = Dog()
bat = NonMarineMammal('Bat')
```

主要内容

- 类和对象
- 封装
- 继承
- **魔术方法**

多态(polymorphism)和鸭子类型(duck type)

多态：同一个方法可以作用于不同类型的对象，具有不同的行为，即接口相同，实现各异

- 静态多态（编译时多态）：依赖于继承与方法重载
 - 父类定义抽象方法，子类实现抽象方法
- 动态多态（运行时多态）：
 - 依赖于鸭子类型：不管对象属于哪个类，只要对象实现了相应的方法，就可以在对象上执行操作
 - 鸭子测试：当看到一只鸟走起来像鸭子、游泳起来像鸭子、叫起来也像鸭子，那么这只鸟就可以被称为鸭子
 - 关注点在于对象的行为，能作什么；而不是关注对象所属的类型

```
class Cat:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        print(f"I am a cat. My name is {self.name}. I am {self.age} years old.")

    def make_sound(self):
        print("Meow")

class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
def info(self):
    print(f"I am a dog. My name is {self.name}. I am {self.age} years old.")

def make_sound(self):
    print("Bark")

cat1 = Cat("Kitty", 2.5)
dog1 = Dog("Fluffy", 4)
for animal in (cat1, dog1):
    animal.make_sound()
    animal.info()
    animal.make_sound()
```

duck_type.py

```
Meow
I am a cat. My name is
Kitty. I am 2.5 years old.
Meow

Bark
I am a dog. My name is
Fluffy. I am 4 years old.
Bark
```

魔术方法(magic method)和运算符重载(operator overloading)

魔术方法也称为dunder (double underscore)方法

- 以双下划线开始，以双下划线结束的特殊方法
- 在调用len/str等内置函数、属性和下标访问以及使用运算符时隐藏调用这些方法

dunder方法	描述
<code>__init__(self, value....)</code>	对应构造函数，初始化实例对象
<code>__str__(self)</code>	对应 <code>str(obj)</code> ，输出友好的字符串
<code>__repr__(self)</code>	对应 <code>repr(obj)</code> ，输出面向程序员的字符串
<code>__format__(self, format_spec)</code>	对应字符串的格式化方法等
<code>__hash__(self)</code>	对应 <code>hash(obj)</code>
<code>__dir__(self)</code>	对应 <code>dir(obj)</code>
<code>__bytes__(self)</code>	对应 <code>bytes(obj)</code>
<code>__bool__(self)</code>	对应 <code>bool(self)</code>
<code>__len__(self)</code>	对应 <code>len(obj)</code>
<code>__getitem__(self, idx)</code>	对应 <code>obj[idx]</code>
<code>__setitem__(self, idx, value)</code>	对应 <code>obj[idx] = value</code>

object类已经包含了一些基本的魔术方法的实现

- `__str__`方法：是调用`__repr__`
- `__format__`方法：调用`__str__`
- 有些类仅实现`__repr__`方法

```
>>> dir(object)
['__class__', '__delattr__',
 '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__',
 '__getattr__', '__gt__',
 '__hash__', '__init__',
 '__init_subclass__', '__le__',
 '__lt__', '__ne__', '__new__',
 '__reduce__', '__reduce_ex__',
 '__repr__', '__setattr__',
 '__sizeof__', '__str__',
 '__subclasshook__']
```

魔术方法

```
class Date:
    def __init__(self, month, day):
        self._month = month
        self._day = day
```

```
d1 = Date(12, 27)
print(d1)
print(repr(d1))
print('{}-{}'.format(d1))
```

```
<__main__.Date object at 0x0000027389502FD0>
<__main__.Date object at 0x0000027389502FD0>
<__main__.Date object at 0x0000027389502FD0>
```

dunder1.py

```
class Date:
    def __init__(self, month, day):
        self._month = month
        self._day = day

    def __repr__(self):
        return '{}-{}'.format(self._month, self._day)

    def __format__(self, format_spec):
        if format_spec == '%d':
            return str(self._day)
        elif format_spec == '%m':
            return str(self._month)
        elif format_spec == '%t' or not format_spec:
            return '{}-{}'.format(self._month, self._day)
        else:
            return ValueError('unknown format spec ' + format_spec)

d2 = Date(12, 27)
print(d2)
print(repr(d2))
print('{0:%d} {0:%m} {0:%t} {0}'.format(d2))
```

```
12-27
12-27
27 12 12-27 12-27
```

运算符重载

比较运算符，在object的缺省实现中：

- `__eq__`和`__ne__`某一个有实现时，可利用一个方法实现另一个(`obj == value` 等价于 `not obj != value`)
- `__lt__`和`__gt__`中某一个有实现时，可利用一个方法实现另一个 (`obj < value` 等价于 `value > obj`)
- `__le__`和`__ge__`中某一个有实现时，可利用另个方法实现另一个
- `__eq__`的缺省实现是 `obj is value`。比较大小的方法的缺省实现是抛出异常

模块functools的**类装饰器**`total_ordering`

- `__lt__`、`__le__`、`__gt__`和`__ge__`中只需要有一个方法有实现即可
- 装饰器会根据已有的实现+ `__eq__`方法自动补齐其他的比较大小的方法

dunder方法	描述
<code>__eq__(self, value)</code>	对应 <code>obj == value</code>
<code>__ne__(self, value)</code>	对应 <code>obj != value</code>
<code>__lt__(self, value)</code>	对应 <code>obj < value</code>
<code>__gt__(self, value)</code>	对应 <code>obj > value</code>
<code>__le__(self, value)</code>	对应 <code>obj <= value</code>
<code>__ge__(self, value)</code>	对应 <code>obj >= value</code>
<code>__contains__(self, value)</code>	对应 <code>self in value</code>

运算符重载: 示例

```
p1 = Point(1, 3)
p2 = Point(3, 1)

for op in '==', '!=', '>', '<',
          '>=', '<=':
    try:
        expr = f'{p1} {op} {p2}'
        result = eval(expr)
        print(expr, result)
    except Exception as e:
        print(type(e), e)
```

```
Point(1,3) == Point(3,1) True
Point(1,3) != Point(3,1) False
Point(1,3) > Point(3,1) False
Point(1,3) < Point(3,1) False
Point(1,3) >= Point(3,1) True
Point(1,3) <= Point(3,1) True
```

class Point:

dunder2.py

```
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point({}, {})'.format(self.x, self.y)

    def length(self):
        import math
        return math.sqrt(self.x ** 2 + self.y ** 2)

    def __eq__(self, value):
        if not isinstance(value, Point):
            return False
        return self.length() == value.length()

    def __lt__(self, value):
        if not isinstance(value, Point):
            return NotImplemented
        return self.length() < value.length()

    def __le__(self, value):
        if not isinstance(value, Point):
            return NotImplemented
        return self.length() <= value.length()
```

运算符重载: 示例

模块functools的**类装饰器**`total_ordering`

- `__lt__`、`__le__`、`__gt__`和`__ge__`中只需要有一个方法有实现即可
- 装饰器会根据已有的实现+ `__eq__`方法自动补齐其他的比较大小的方法

```
import functools
@functools.total_ordering
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point({}, {})'.format(self.x, self.y)

    def length(self):
        import math
        return math.sqrt(self.x ** 2 + self.y ** 2)

    def __eq__(self, value):
        if not isinstance(value, Point):
            return False
        return self.length() == value.length()

    def __lt__(self, value):
        if not isinstance(value, Point):
            return NotImplemented
        return self.length() < value.length()
```

运算符重载

dunder4.py

算术运算符: 产生一个新的对象

dunder方法	描述
<code>__add__(self, other)</code>	<code>obj + other</code>
<code>__sub__(self, other)</code>	<code>obj - other</code>
<code>__mul__(self, other)</code>	<code>obj * other</code>
<code>__matmul__(self, other)</code>	<code>obj @ other</code> , 矩阵乘法
<code>__truediv__(self, other)</code>	<code>obj / other</code>
<code>__floordiv__(self, other)</code>	<code>obj // other</code>
<code>__mod__(self, other)</code>	<code>obj % other</code>
<code>__divmod__(self, other)</code>	<code>divmod(obj, other)</code>
<code>__pow__(self, other)</code>	<code>obj ** other</code>

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point({}, {})'.format(self.x, self.y)

    def __add__(self, other):
        if isinstance(other, int):
            return Point(self.x + other, self.y + other)
        elif isinstance(other, Point):
            return Point(self.x + other.x, self.y + other.y)
        else:
            return NotImplemented

p1 = Point(1, 3)
p2 = Point(2, 1)
print(p1 + p2) # 两个Point对象相加
print(p1 + 4) # Point与整数对象相加
print(p1 + 4.0)
```

```
Point(3,4)
Point(5,7)
Traceback (most recent call last):
print(p1 + 4.0)
TypeError: unsupported operand type(s) for +: 'Point' and 'float'
```

运算符重载

- 算术运算符：由左操作数作为主导，如 $a + b$ 相当于 `a.__add__(b)`
- 反射算术运算符(reflected arithmetic operator)：
 - 通过右操作数调用反射算术运算符，计算 $a + b$ ，调用 `b.__radd__(a)`
 - 左操作数的常规算术运算符返回 `NotImplemented` 时调用
- $p + 4$ 等价于 `p.__add__(4)`
- $4 + p$ ：
 - 首先调用 `(4).__add__(p)`
 - 如果整数的加法方法返回 `NotImplemented`，尝试调用 `p.__radd__(4)`

```
Point(3,4)
Point(5,7)
Point(5,7)
```

dunder5.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point({}, {})'.format(self.x, self.y)

    def __add__(self, other):
        if isinstance(other, int):
            return Point(self.x + other, self.y +
other)
        elif isinstance(other, Point):
            return Point(self.x + other.x, self.y +
other.y)
        else:
            return NotImplemented

    def __radd__(self, other):
        return self.__add__(other)

p1 = Point(1, 3)
p2 = Point(2, 1)
print(p1 + p2)
print(p1 + 4)
print(4 + p1)    # 调用反射运算符实现
```

运算符重载

复合赋值运算符(augmented assignment)

- `__iadd__(self, other)`对应obj += other

```
def __iadd__(self, other):
    if isinstance(other, int):
        self.x += other
        self.y += other
        return self
    elif isinstance(other, Point):
        self.x += other.x
        self.y += other.y
        return self
    else:
        return NotImplemented
```

```
p = Point(1, 3)
print(id(p), p)
p += 3
print(id(p), p)
```

dunder5.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Point({}, {})'.format(self.x, self.y)

    def __add__(self, other):
        if isinstance(other, int):
            return Point(self.x + other, self.y +
other)
        elif isinstance(other, Point):
            return Point(self.x + other.x, self.y +
other.y)
        else:
            return NotImplemented

    def __radd__(self, other):
        return self.__add__(other)
```

```
2716315766304 Point(1,3)
2716315766304 Point(4,6)
```